

Preston-Check — Operator Runbook

Deployed URLs, resource IDs, deploy procedures, emergency response

Preston-Check Maintainers

2026-05-06

Operator Runbook

The single-page reference for everything deployed. Bookmark this file and the page-rendered version at `https://admin.preston-check.com/` (once admin portal includes it).

This file is intentionally in the public repo because it contains zero credentials — only public URLs, non-sensitive resource identifiers, and procedures. The actual secrets (Cloudflare API token, license signing private key) live only in repo secrets and the operator’s hardware-bound machine.

Production URLs

SURFACE	URL	HOSTED ON	AUTH
Public landing	<code>https://preston-check.com/</code>	GitHub Pages (custom domain via Route 53 A records)	none
Legal pages	<code>https://preston-check.com/{terms,privacy,refund}.html</code>	GitHub Pages	none
GitHub Pages mirror	<code>https://preston-check.github.io/preston-check/</code>	GitHub Pages (default subdomain)	none
Customer portal	<code>https://app.preston-check.com/</code>	Cloudflare Pages (<code>preston-check-customer</code> project)	magic-link via auth Worker

SURFACE	URL	HOSTED ON	AUTH
Admin portal	<code>https://admin.preston-check.com/</code>	Cloudflare Pages (<code>preston-check-admin</code> project)	Cloudflare Access — operator email + 6-digit PIN
Auth Worker	<code>https://preston-check-auth.preston-check-edge.workers.dev/</code>	Cloudflare Workers	per-endpoint (POST <code>/request-code</code> , <code>/verify-code</code> , GET <code>/me</code> , POST <code>/logout</code>)
Billing Worker	<code>https://preston-check-billing.preston-check-edge.workers.dev/</code>	Cloudflare Workers	per-endpoint (POST <code>/checkout</code> , <code>/webhook</code> , <code>/billing-portal</code> , <code>/license</code>)
Telemetry Worker	<code>https://preston-check-telemetry.preston-check-edge.workers.dev/</code>	Cloudflare Workers	none (public POST endpoint by design)

`*.pages.dev` mirrors (also serve, but use the custom domains in production): - `https://preston-check-admin.pages.dev/` - `https://preston-check-customer.pages.dev/`

Cloudflare resources

RESOURCE	TYPE	ID / NAME	NOTES
Pages project	Customer portal	<code>preston-check-customer</code>	Auto-deploys from <code>web/custom</code> on each master push
Pages project	Admin portal	<code>preston-check-admin</code>	Auto-deploys from <code>web/admin/</code>
Workers script	Auth	<code>preston-check-auth</code>	Magic-link authentication; deploys from <code>workers/auth/</code>
Workers script	Billing	<code>preston-check-billing</code>	Stripe Checkout + webhook + port + license issuance; deploys from <code>workers/billing/</code>

RESOURCE	TYPE	ID / NAME	NOTES
Workers script	Telemetry collector	<code>preston-check-telemetry</code>	Standalone Worker; deploys from <code>workers/telemetry/</code>
D1 database	Auth	<code>preston-check-auth</code> (<code>fb99b043-dd02-4a84-b264-07ae4aa766d1</code>)	accounts + sessions tables. Schema in <code>workers/auth/schema.sql</code> .
D1 database	Billing	<code>preston-check-billing</code> (<code>abeeb20d-935f-4551-8d2e-a8eaecee8c0b</code>)	customers + subscriptions + invoice + webhook_events. Schema in <code>workers/billing/schema.sql</code>
D1 database	Telemetry storage	<code>preston-check-telemetry</code> (<code>e206e1e4-1c78-4a5e-a983-bc47104d1b3c</code>)	ENAM region. Schema in <code>workers/telemetry/schema.sc</code>
KV namespace	Auth codes	<code>CODES</code> (<code>29537c75a6424519b1ccfe748aae7fd9</code>)	6-digit sign-in codes (10-min TTL per-email rate limits)
KV namespace	Billing nonces	<code>STATE</code>	Short-lived idempotency keys + per email checkout rate limits
KV namespace	Telemetry aggregations	<code>AGGREGATE</code> (<code>330983ec5b464dab8ae2f338d40512f5</code>)	Used by Worker + Pages Function
R2 bucket	D1 backup target	<code>preston-check-d1-backups</code>	Created on first nightly run; nightly auth + billing dumps under <code>auth/</code> and <code>billing/</code> prefixes
Workers subdomain	Account	<code>preston-check-edge.workers.dev</code>	Renamed from default to be anonymity-clean
Access app	Operator gate	"Preston-Check Admin" → <code>admin.preston-check.com</code>	One-time PIN to operator email; 24 sessions
Pages bindings	DB + AGGREGATE + env	On customer-portal project (production env)	Set via Cloudflare API; visible in dashboard → Pages → preston-check-customer → Settings → Functions

DNS records

In Route 53 hosted zone `preston-check.com.` (zone ID `Z05163633TTJD7W45GALS`). The domain itself is registered at Route 53, separate from the hosted zone.

RECORD	TYPE	VALUE	TTL	PURPOSE
preston-check.com.	A (x4)	185.199.108-111.153	default	GitHub Pages apex
admin.preston-check.com.	CNAME	preston-check-admin.pages.dev	300	Admin portal
app.preston-check.com.	CNAME	preston-check-customer.pages.dev	300	Customer portal
preston-check.com.	MX	10 inbound-smtp.us-east-1.amazonaws.com.	300	Inbound mail to SES
preston-check.com.	TXT	v=spf1 include:amazonses.com -all	300	SPF (SES outbound + inbound)
_dmarc.preston-check.com.	TXT	v=DMARC1; p=none; rua=mailto:postmaster@preston-check.com	300	DMARC reporting
*._domainkey.preston-check.com.	CNAME (x3)	(DKIM tokens issued by SES)	300	DKIM signing for outbound

GitHub repository

	VALUE
Org / repo	preston-check/preston-check
Default branch	master
Visibility	Public (since 2026-05-04)
Latest tag	v1.7.10
License	Apache 2.0
Homebrew tap	preston-check/homebrew-tap (separate repo)
GitHub Action	preston-check/scan-action (separate repo, still a placeholder — not yet referenced by any active workflow; reserved for the eventual published action)

	VALUE
Container registry	GitHub Container Registry (<code>ghcr.io/preston-check/preston-check</code>) — replaced Docker Hub on 2026-05-04

Repository secrets

Set under repo Settings → Secrets and variables → Actions. Never logged, never echoed in workflow output. Always pipe values via `printf '%s'`, never `echo` (echo adds a trailing newline that breaks SigV4 / token validation).

SECRET	USED BY	NOTES
<code>CLOUDFLARE_API_TOKEN</code>	all CF deploy workflows	Scoped to Cloudflare Pages + Workers + KV + D1 + R2; rotate via dashboard → My Profile → API Tokens → Roll
<code>CLOUDFLARE_ACCOUNT_ID</code>	all CF deploy workflows	Not a credential — Cloudflare account identifier
<code>SES_AWS_ACCESS_KEY_ID</code>	auth-deploy.yml	Pushed into auth Worker as runtime secret; IAM user <code>preston-check-ses</code> scoped to <code>ses:SendEmail</code> on <code>preston-check.com</code> only
<code>SES_AWS_SECRET_ACCESS_KEY</code>	auth-deploy.yml	Companion to above
<code>HOME BREW_TAP_TOKEN</code>	release.yml (homebrew job)	Optional — if absent, formula is bumped manually. PAT with write access to <code>preston-check/homebrew-tap</code>

Worker secrets (set via `wrangler secret put`)

These live inside Cloudflare and never appear in the repo or in GH Actions logs. Inspect with `wrangler secret list --name <worker>`.

WORKER	SECRET	PURPOSE
<code>preston-check-auth</code>	<code>SES_AWS_ACCESS_KEY_ID</code> , <code>SES_AWS_SECRET_ACCESS_KEY</code>	SES SigV4 sending. Synced from GH secrets on each auth deploy.
<code>preston-check-auth</code>	<code>RESEND_API_KEY</code> (optional)	Fallback when SES is misconfigured.
<code>preston-check-auth</code>	<code>SESSION_SECRET</code> (optional)	Salt for <code>ip_hash</code> column. Defaults to empty if unset.
<code>preston-check-billing</code>	<code>STRIPE_SECRET_KEY</code>	<code>sk_live_...</code> — billing API.
<code>preston-check-billing</code>	<code>STRIPE_WEBHOOK_SECRET</code>	<code>whsec_...</code> — verifies webhook signature.
<code>preston-check-billing</code>	<code>STRIPE_PRICE_PRO_PER_REPO</code> , <code>STRIPE_PRICE_PRO_UNLIMITED</code>	Stripe price IDs from the live products.
<code>preston-check-billing</code>	<code>LICENSE_SIGNING_KEY</code>	PEM-encoded Ed25519 private key. The corresponding public key is checked into <code>lib/license_saas_pubkey.pem</code> and the runner verifies licenses against either it or <code>lib/license_pubkey.pem</code> (operator-issued).

GitHub Actions workflows

WORKFLOW	TRIGGER	PURPOSE
<code>pages.yml</code>	Push to master touching <code>web/landing/**</code>	Deploys public landing (incl. legal pages) to GitHub Pages
<code>admin-pages.yml</code>	Push touching <code>web/admin/**</code> or shared assets	Deploys admin portal to Cloudflare Pages
<code>customer-pages.yml</code>	Push touching <code>web/customer/**</code> or shared assets	Deploys customer portal to Cloudflare Pages
<code>auth-deploy.yml</code>	Push touching <code>workers/auth/**</code>	Syncs SES secrets into Worker, then deploys Worker

WORKFLOW	TRIGGER	PURPOSE
<code>billing-deploy.yml</code>	Push touching <code>workers/billing/**</code>	Deploys billing Worker
<code>telemetry-deploy.yml</code>	Push touching <code>workers/telemetry/**</code>	Deploys telemetry Worker
<code>d1-backup.yml</code>	Daily 03:00 UTC + manual dispatch	Exports auth + billing D1 to R2 bucket <code>preston-check-d1-backups</code>
<code>release.yml</code>	Tag push (<code>v*</code>)	Build tarball + sha256 + GitHub Release; GHCR image; Homebrew bump
<code>test.yml</code>	Every push + PR	Shell syntax check + smoke scans + framework filter test
<code>lint-community.yml</code>	PR touching <code>checks/community/**</code>	Shellcheck + lint-check.sh on community contributions
<code>threat-intel-sync.yml</code>	Mondays 09:00 UTC + manual dispatch	Pulls fintech-relevant CVEs from NIST NVD; opens PR with draft checks
<code>preston-check.yml</code>	Every push + PR to master	Self-audit: runs the in-repo <code>./preston-check.sh --full -airgap --ci-soft</code> and uploads the markdown report as an artifact. Soft mode means findings don't fail the build (yet); promote to <code>--ci</code> once the catalog is tuned for self-application.

Health checks

Confirms every public surface is up:

```
for u in \
  https://preston-check.com/ \
  https://preston-check.com/terms.html \
  https://preston-check.com/privacy.html \
  https://preston-check.com/refund.html \
  https://app.preston-check.com/ \
```

```
https://admin.preston-check.com/; do
printf "%-55s " "$u"
curl -so /dev/null -w "%{http_code}\n" "$u" --max-time 6
done
```

Expected codes: `200 200 200 200 200 302` (admin → 302 because Access redirect).

End-to-end Worker probe (no payment needed):

```
# Auth Worker - request a code (will actually email you, so use a throwaway)
curl -sS -X POST https://preston-check-auth.preston-check-edge.workers.dev/request-code \
-H 'Content-Type: application/json' \
-d '{"email":"smoke@preston-check.com"}' # expect: {"sent":true,"via":"ses"}

# Billing Worker - checkout returns a Stripe URL
curl -sS -X POST https://preston-check-billing.preston-check-edge.workers.dev/checkout \
-H 'Content-Type: application/json' -H 'Origin: https://app.preston-check.com' \
-d '{"plan":"pro_per_repo","email":"smoke@preston-check.com","org_name":"Smoke"}'
# expect: {"url":"https://checkout.stripe.com/c/pay/cs_live_...","id":"cs_live_..."}

# Inbound mail probe - sends to support@ and verifies S3 delivery
AWS_PROFILE=dev AWS_REGION=us-east-1 aws sesv2 send-email \
--from-email-address 'Preston-Check <preston@preston-check.com>' \
--destination 'ToAddresses=support@preston-check.com' \
--content 'Simple={Subject={Data=Probe,Charset=UTF-8},Body={Text={Data=ping,Charset=UTF-8}}}'
sleep 8
AWS_PROFILE=dev AWS_REGION=us-east-1 aws s3 ls s3://preston-check-inbound-
mail/incoming/ | tail -1
```

AWS resources (account 356697059290, region us-east-1)

The AWS account is managed under the operator’s existing dev profile (`AWS_PROFILE=dev`). Resources are scoped to preston-check.com so they can be migrated to a Preston-Check-only account later without unwinding personal email or other domains.

RESOURCE	IDENTIFIER	PURPOSE
SES domain identity	<code>preston-check.com</code>	Outbound + inbound. DKIM_SUCCESS, ProductionAccess enabled (85,500/day quota)
IAM user	<code>preston-check-ses</code>	Worker-side SES sending. Inline policy <code>PrestonCheckSESSendOnly</code> allows <code>ses:SendEmail</code> / <code>ses:SendRawEmail</code>

RESOURCE	IDENTIFIER	PURPOSE
		on the preston-check.com identity ARN only
S3 bucket	preston-check-inbound-mail	Inbound mail dropbox. Public access blocked. Bucket policy allows s3:PutObject only from ses.amazonaws.com with AWS:SourceAccount 356697059290
SES rule set	INBOUND_MAIL (active)	Existing rule set; we added rule support-to-s3 matching recipients ["preston-check.com"] with action S3Action(BucketName=preston-check-inbound-mail, ObjectKeyPrefix=incoming/)
Route 53 hosted zone	Z05163633TTJD7W45GALS for preston-check.com.	All DNS for the domain

Inbound mail flow: support@preston-check.com → MX → SES → receipt rule → S3 incoming/ prefix. Polling vs push is currently polling (operator runs aws s3 ls periodically); a Lambda forwarder is the next iteration if volume warrants.

Stripe live mode

Stripe is the source of truth for billing state; D1 holds a denormalised mirror populated by the webhook handler.

ITEM	VALUE
Mode	Live
API version pinned via header	2024-04-10 (set on calls that use coupon parameter)
Webhook endpoint	https://preston-check-billing.preston-check-edge.workers.dev/webhook
Webhook events subscribed	checkout.session.completed , customer.subscription.{created,updated,deleted} , invoice.{paid,payment_failed,finalized}
Webhook payload format	snapshots (set explicitly when creating the endpoint)

ITEM	VALUE
Customer Portal	Configured; opens via <code>POST /billing-portal</code> returning a session URL

Promo codes (live):

CODE	DISCOUNT	USE CASE
<code>FOUNDING100</code>	100% off, first 10 redemptions	Founding-customer / smoke-test path with no card charge
<code>EARLY50</code>	50% off, first 6 months	Early adopter outreach
<code>CONFERENCE</code>	50% off, first 3 months	Conference / event attendees
<code>SOC2READY</code>	30% off, first 12 months	SOC 2 / compliance ICP
<code>PARTNER25</code>	25% off forever	Partner referral incentive

Deploy procedures

Standard release flow

```
# 1. Bump PRESTON_VERSION in preston-check.sh
# 2. Update CHANGELOG.md
# 3. Commit
git commit -m "vX.Y.Z: ..."
# 4. Tag + push
git tag -a vX.Y.Z -m "vX.Y.Z: short summary"
git push origin master --tags
```

The `release.yml` workflow then auto-creates the GitHub Release with `install.sh` + tarball + sha256 sidecar. If `HOMEbrew_TAP_TOKEN` is set, the homebrew formula is also bumped automatically.

Manual homebrew bump (if auto-bump unavailable)

```
SHA=$(curl -fsSL https://github.com/preston-check/preston-check/releases/download/vX.Y.Z/preston-check-X.Y.Z.tar.gz.sha256 | awk '{print $1}')
git -C /tmp/homebrew-tap pull
# Edit Formula/preston-check.rb: url, sha256, version
git -C /tmp/homebrew-tap commit -am "preston-check X.Y.Z"
git -C /tmp/homebrew-tap push
```

One-off Cloudflare admin tasks

```
export CLOUDFLARE_API_TOKEN="..." # from repo secrets, or operator's local
export CLOUDFLARE_ACCOUNT_ID="..."

# List all Pages projects
curl -s -H "Authorization: Bearer $CLOUDFLARE_API_TOKEN" \
  "https://api.cloudflare.com/client/v4/accounts/$CLOUDFLARE_ACCOUNT_ID/pages/projects" \
  | python3 -m json.tool

# Query the telemetry D1 directly
wrangler d1 execute preston-check-telemetry --remote --command "SELECT count(*) FROM scans"

# Tail the customer-portal Function logs
wrangler pages deployment tail --project-name=preston-check-customer
```

Emergency response

Compromised Cloudflare API token

1. `https://dash.cloudflare.com/profile/api-tokens` → click "..." next to the token → **Revoke**
2. Create a new token with the same scopes (Cloudflare Pages: Edit, Workers Scripts: Edit, Account Settings: Read, Workers KV Storage: Edit, D1: Edit)
3. `gh secret set CLOUDFLARE_API_TOKEN --repo preston-check/preston-check` → paste new value
4. Verify next deploy still succeeds (push a no-op or run `gh workflow run`)

Compromised license signing key

There are two keypairs:

The operator-side key at `~/.preston-check/keys/private.pem` (hardware-bound) signs licenses issued out-of-band by the operator. The corresponding public half is `lib/license_pubkey.pem` checked into the repo.

The SaaS-side key is held as Worker secret `LICENSE_SIGNING_KEY` on the billing Worker and signs licenses issued automatically by the `/license` endpoint after Stripe payment. The corresponding public half is `lib/license_saas_pubkey.pem`.

If either is compromised:

1. Generate a new keypair (`openssl genpkey -algorithm ED25519 -out new_private.pem; openssl pkey -in new_private.pem -pubout -out new_public.pem`)
2. Replace the corresponding public PEM in the repo (`lib/license_pubkey.pem` for operator, `lib/license_saas_pubkey.pem` for SaaS)

3. For the SaaS key: `cat new_private.pem | wrangler secret put LICENSE_SIGNING_KEY --name preston-check-billing`
4. Re-issue every active license against the new key
5. Push a new release (any vX.Y.Z) so customers get the updated public key on their next `brew upgrade` / `git pull`
6. Old licenses no longer verify; old private key file gets deleted

Compromised SES IAM key

The IAM user `preston-check-ses` keys are stored both as GH Actions secrets (`SES_AWS_ACCESS_KEY_ID` , `SES_AWS_SECRET_ACCESS_KEY`) and as Worker secrets on the auth Worker. Rotation order matters — never delete-before-replace:

```
# 1. Create new key
AWS_PROFILE=dev aws iam create-access-key --user-name preston-check-ses
# 2. Update GH secrets (printf, not echo)
printf '%s' "$NEW_AKID" | gh secret set SES_AWS_ACCESS_KEY_ID -R preston-check/preston-check
printf '%s' "$NEW_SECRET" | gh secret set SES_AWS_SECRET_ACCESS_KEY -R preston-check/preston-check
# 3. Trigger auth-deploy to push new keys into the Worker
gh workflow run auth-deploy.yml -R preston-check/preston-check --ref master
# 4. Smoke test
curl -sS -X POST https://preston-check-auth.preston-check-edge.workers.dev/request-code \
  -H 'Content-Type: application/json' -d '{"email":"smoke@preston-check.com"}'
# 5. Only after smoke test passes – deactivate, then delete old key
AWS_PROFILE=dev aws iam update-access-key --user-name preston-check-ses --access-key-id $OLD_AKID --status Inactive
AWS_PROFILE=dev aws iam delete-access-key --user-name preston-check-ses --access-key-id $OLD_AKID
```

Customer portal compromised / data leak

Customer portal currently has no real customer data — mock skeleton only. If real data has been migrated and a leak is suspected:

1. Cloudflare dashboard → Workers & Pages → preston-check-customer → temporarily delete or pause the deployment
2. Investigate logs via Cloudflare Pages → Functions → Logs
3. Rotate any database credentials referenced in the project bindings
4. Issue customer notifications via the published security disclosure policy in `SECURITY.md`

Lost operator access to admin portal

Cloudflare Access is the gate. If the operator's email is no longer accessible:

1. Cloudflare Zero Trust dashboard → Access → Applications → Preston-Check Admin → edit the policy
2. Change the Allow rule's email to the new operator address
3. Save — next visit to `admin.preston-check.com` prompts for the new email

If the entire Zero Trust account is locked:

1. Use the backup recovery contact email if one is configured
2. If not, follow Cloudflare's account recovery flow at `https://dash.cloudflare.com/sign-in`
3. The deploy infrastructure continues working without admin portal access (admin is only needed for operator UI; deploys go through GitHub Actions independently)

Monitoring & observability

What's currently in place:

- **Cloudflare Workers Logs (Live Logs)** — both auth and billing Workers wrap their `fetch` handler in a global try/catch that emits structured JSON on errors. View at dashboard → Workers & Pages → Worker → Logs. Bodies and the Stripe signature header are never logged. Each 500 includes a `request_id` returned to the client for correlation.
- **wrangler tail** — for live tailing during incidents: `wrangler tail preston-check-auth --format pretty` (or `preston-check-billing`).
- **Cloudflare Pages dashboard** — deploy history, function logs, custom domain status. Per-project at `dash.cloudflare.com/<account>/pages/view/<project>`.
- **GitHub Actions tab** — every workflow run, with logs. `https://github.com/preston-check/preston-check/actions`
- **D1 query interface** — `wrangler d1 execute preston-check-{auth,billing,telemetry} --remote --command "..."` for ad-hoc queries.
- **D1 nightly backups** — `.sql` dumps in `s3://preston-check-d1-backups/{auth,billing}/YYYYMMDDTHMMSSZ.sql`. Restore via `wrangler d1 execute --remote --file <dump>` after creating an empty target DB.
- **SES sending stats** — `aws ses get-send-statistics` for delivery / bounce / complaint counters.
- **Cloudflare Web Analytics** — not yet enabled; the bound projects' "Web analytics" tab in the Pages dashboard activates it.

What's not in place yet:

- Sentry / Honeycomb / Datadog — Cloudflare Workers Logs is sufficient at current volume; revisit when 500s/day exceeds what's manageable in the Live Logs view
- Public status page at `https://status.preston-check.com/`
- Daily KPI digest email
- S3-to-email forwarder for the support inbox (currently polled manually via `aws s3 ls`)

Anonymity / pseudonymity notes

The current Cloudflare account is registered to the operator's pre-pseudonym email. Long-term hygiene per `docs/strategy/anonymity-and-mystique.md` :

1. Form a Wyoming LLC ("Catalog Holdings LLC" or similar)
2. Register a fresh Cloudflare account using `operator@<llc-domain>` or `preston@preston-check.com`
3. Migrate Pages projects + Worker + D1 + KV to the new account
4. Update `CLOUDFLARE_ACCOUNT_ID` repo secret with the new account's ID
5. Delete the old account once everything is verified on the new one

Until then: the workers.dev subdomain has been renamed from a personal one to `preston-check-edge.workers.dev` (anonymity-clean), and operator-facing emails are role-based (`hello@`, `security@`, `support@`, `press@`, `preston@` all forward to the operator's actual mailbox via Cloudflare Email Routing or equivalent).

Where things live (file-system map)

```

/web/landing/      - public landing page (deploys to preston-check.com via GitHub Pages)
/web/landing/{terms,privacy,refund}.html - legal pages, footer-linked
/web/landing/icons/ - proprietary 23-icon SVG set, shared by all surfaces
/web/landing/assets/ - logomark, wordmark, score-badge, watermark
/web/admin/        - admin portal SPA (deploys to admin.preston-check.com via Cloudflare Pages)
/web/customer/     - customer portal SPA (deploys to app.preston-check.com via Cloudflare Pages)

/workers/auth/      - magic-link auth Worker (preston-check-auth)
/workers/billing/   - Stripe Checkout + webhook + portal + license-issuance Worker (preston-check-billing)
/workers/telemetry/ - telemetry collector Worker (preston-check-telemetry)

/lib/              - runner libraries (license, telemetry, ai_analyze, ai_autofix, branding, oss_detection)
/lib/license_pubkey.pem - operator-side public key
/lib/license_saas_pubkey.pem - SaaS-side public key (companion to LICENSE_SIGNING_KEY Worker secret)
/checks/           - catalog (294 checks total: core + community/{verified,accepted,proposed})
/modules/smart-contract-audit/ - separate runner for deep contract audits
/tools/            - operator scripts (issue-license, setup-signing-key, sync-threat-intel, integrations, build-docs)
.github/workflows/ - twelve CI workflows (pages, admin-pages, customer-pages, auth-deploy, billing-deploy, telemetry-deploy, d1-backup, release, test, lint-community, threat-intel-sync, preston-check)
/docs/            - design docs (architecture, portals-and-kpis, strategy/, sales/, etc.)
/docs/sales/       - playbook, promo-codes, first-five-targets exercise

```

Updating this runbook

This file lives at `docs/operator-runbook.md` in the public repo. Keep it current — every time a new resource is provisioned, a URL changes, or an emergency procedure needs adjustment, update this file in the same commit. The file is intentionally short on prose and long on tables so it stays scannable at 3am during an incident.